

Parallel Numerical Library

One numerical library, seven execution backends,
and an honest account of what each one costs

Olajide Badejo

August 2026

Abstract

This report describes a C++23 numerical library in which twelve iterative solvers are written once, against a single problem interface, and run unchanged over seven execution backends: serial, OpenMP, POSIX threads, a `std::jthread` pool, MPI, hybrid MPI with threads, and CUDA. The design makes identical numerics across backends a tested invariant rather than an assumption, and in its strong form: with the deterministic reduction mode every shared memory backend at every worker count produces bit identical iterates, and the GPU sweeps are bit identical to the CPU too.

That invariant is what makes the measurements comparable. Because every backend computes the same values, the differences between them are attributable to the execution model and to nothing else.

The convergence behaviour of the solver zoo is checked against closed form theory rather than asserted: the Jacobi spectral radius against $\cos(\pi h)$, the ratio of two between Jacobi and Gauss Seidel iteration counts, Young's optimal relaxation factor as a genuine minimum, and the change of growth order from $O(n^2)$ to $O(n)$ that optimal relaxation buys.

The CPU versus GPU comparison is constructed so that the hardware does not decide the argument. Iteration counts, which are hardware free, are reported first. Efficiency against each device's own measured STREAM bandwidth is reported second. Absolute time is reported last and labelled as a property of one particular pair of devices. The resulting speedup is then decomposed into the part that is the memory system and the part that is the implementation.

Contents

1	Introduction	4
1.1	The question	4
1.2	The invariant that makes it work	4
1.3	The solver zoo, and why it is a family	5
1.4	A comparison the hardware does not decide	5
1.5	What the environment allowed, and what it did not	5
1.6	How to read this report	6
2	Mathematical background	7
2.1	The splitting framework	7
2.2	The model problem	7
2.3	Richardson iteration	8
2.4	Jacobi	8
2.5	Gauss Seidel, and the factor of two	8
2.6	Successive over relaxation	9
2.7	Red black ordering	9
2.8	Block methods	10
2.9	Conjugate gradient	10
2.10	The numerics module	11
3	Parallelisation design	12
3.1	One interface	12
3.1.1	Why the callbacks are chunk level	12
3.1.2	What backends may not do	12
3.2	Determinism as a design decision	13
3.3	Contraction, and a claim that nearly was not true	13
3.4	What each backend does underneath	14
3.4.1	Shared memory	14
3.4.2	Distributed memory	14
3.4.3	Sequential dependence across ranks	14
3.5	The CUDA boundary	15

3.6	Thread pinning under a hypervisor	15
4	Comparison methodology	16
4.1	Why the naive comparison is misleading	16
4.2	The protocol	16
4.2.1	First: iterations to tolerance	16
4.2.2	Like against like	17
4.2.3	Second: bytes per unknown, counted	17
4.2.4	Third: each device's own bandwidth, measured	17
4.2.5	The comparable number	17
4.2.6	Last: absolute time	17
4.3	Measuring at sizes where methods do not converge	17
4.4	Experimental hygiene	18
4.5	What this comparison does not establish	18
5	Results	20
5.1	The machine	20
5.2	Iterations to tolerance	20
5.2.1	Agreement with theory	20
5.3	Backend cost on identical work	23
5.4	Scaling and the core knee	23
5.4.1	What could and could not be established	23
5.4.2	The knee is not where the objective expected it	25
5.5	Pinning	25
5.6	The cost of determinism	25
5.7	Scheduling	27
5.8	Distributed scaling and the communication fraction	27
5.9	The CPU versus GPU comparison	27
5.9.1	Decomposing the speedup	28
5.9.2	The crossover, derived	29
6	Discussion	30
6.1	What the equivalence invariant bought	30
6.2	The methods that do not parallelise, and why they are still here	30
6.3	Where the environment set the limits	31
6.4	Things that were wrong before they were right	31
6.5	What a reader should take away	32
7	Conclusion	33
7.1	What was built	33

7.2	What was established	33
7.3	What was not established	34
7.4	Further work	34

Chapter 1

Introduction

1.1 The question

The question worth asking about parallel programming models is not whether OpenMP is fast. It is what each model costs on identical work. That question is hard to answer honestly because the usual comparison changes two things at once: a study that reimplements an algorithm per model ends up measuring the care that went into each reimplementation, and one that compares different algorithms on different devices ends up measuring the algorithms.

This project answers it by construction. One numerical library, twelve iterative solvers written exactly once against a single problem interface, and seven execution backends behind a single parallelism interface. A solver does not know which backend it is running on. A backend does not know which solver it is running. What varies between measurements is therefore the execution model, and that is enforced by a test rather than promised in a paragraph.

1.2 The invariant that makes it work

The design commits to something stronger than the usual claim of numerical agreement. With the deterministic reduction mode:

- every shared memory backend, at every worker count, produces *bit identical* iterates, not merely close ones;
- the GPU sweeps produce bit identical values to the CPU as well;
- across MPI rank counts, results agree to reduction tolerance, and the reason they cannot be bit identical is documented rather than hidden.

This is achievable because floating point addition is not associative and the library therefore refuses to let the association vary. The chunk grid used by every reduction is fixed by the problem size alone and never by the worker count, so the partial sums are always combined in the same order regardless of which worker produced which one or when. Chapter 3 explains the mechanism; the equivalence suite asserts exact equality across twelve solvers, two problem families, four backends and seven worker counts.

Getting there required disabling fused multiply add contraction on both the host and the device. The SOR update has the shape $ab + cd$, which a compiler is free to fuse, and the host was fusing it while the device was not. A claim of identical numerics that holds only when the compiler

happens not to contract is a much weaker claim than it appears, so contraction is off on both sides. On kernels that are bandwidth bound this costs nothing measurable.

1.3 The solver zoo, and why it is a family

Nine methods are required by the objectives and three more earn their place. Eleven of the twelve are one family: every classical stationary method is a choice of matrix splitting $A = M - N$, and the differences between Richardson, Jacobi, the Gauss Seidel variants, SOR and the block methods are entirely differences in the choice of M . Chapter 2 presents them that way, derives each in a few lines from the common framework, and cites the source for each convergence result.

Conjugate gradient is the twelfth and is deliberately not in that list. It is not stationary, its optimality argument is different in kind, and presenting it as the contrast is what makes the family visible as a family.

Every convergence claim in this report is checked as a measurement. The Jacobi spectral radius is compared against $\cos(\pi h)$; the ratio of Jacobi to Gauss Seidel iteration counts is compared against the factor of two that Young's theory predicts; the closed form optimal relaxation factor is tested as a minimum by perturbing it in both directions; and the change of growth order from $O(n^2)$ to $O(n)$ is demonstrated by measuring how the counts grow as the grid doubles.

1.4 A comparison the hardware does not decide

A five point stencil sweep performs three memory operations and about six floating point operations per unknown. It is bandwidth bound on every machine ever built. A ratio of runtimes between a CPU and a GPU on such a kernel is therefore, to within a small correction, a ratio of memory bandwidths wearing the costume of a statement about parallel programming.

Chapter 4 sets out the protocol that avoids this. Iteration counts come first because they are properties of the mathematics and transfer to any machine. Efficiency against each device's own measured STREAM triad bandwidth comes second, because it is dimensionless and says how well a device is being used rather than which device is larger. Absolute time comes last, labelled as a property of this specific pair of devices. The measured speedup is then decomposed into the factor that is the memory system and the factor that is the implementation, so a reader can see which is which.

Like is compared against like throughout. Natural ordering Gauss Seidel cannot run on a GPU at all, because it is sequentially dependent; the comparison therefore pairs GPU Jacobi against CPU Jacobi and GPU red black Gauss Seidel against CPU red black Gauss Seidel, and reports the cost of the red black reordering separately so that it is charged to both sides equally.

1.5 What the environment allowed, and what it did not

The target machine has an Intel Core i7-14700K with eight performance cores and twelve efficiency cores, and everything runs inside WSL2. The guest does not pass the heterogeneity through: it reports fourteen uniform cores, identical cache sizes on every logical processor, and no hybrid feature flag. Thread affinity binds to a guest virtual processor that the hypervisor may place on any host core.

Rather than assume a mapping that cannot be verified from inside the guest, the library measures. A probe times an identical kernel on each logical processor and reports a two group

split only when the throughputs separate by a margin that dominates the noise. The core knee that the objectives ask for is taken from the aggregate scaling curve, which depends only on how many cores are engaged and not on which, and therefore survives virtualisation. Section 5.4 reports both what could and what could not be established.

1.6 How to read this report

Chapter 2 derives the solver family and states the convergence theory. Chapter 3 describes the backend interface, what each implementation does underneath, and the communication analysis for the distributed backends. Chapter 4 is the comparison protocol, written to be difficult to misquote. Chapter 5 contains the measurements. Chapter 6 interprets them, including the parts that did not come out as expected. Chapter 7 summarises what the work established and what it did not.

Every number in Chapter 5 is generated from `experiments/results/summary.csv` by a script, and every row of that file carries the backend, worker count, pinning policy, seed and commit hash that produced it. A number that has not been measured reads `pending`.

Chapter 2

Mathematical background

2.1 The splitting framework

Let A be nonsingular and write $A = M - N$ with M invertible and cheap to invert. Then $Ax = b$ is equivalent to $Mx = Nx + b$, and the associated iteration is

$$x_{k+1} = M^{-1}(Nx_k + b) = x_k + M^{-1}r_k, \quad r_k = b - Ax_k. \quad (2.1)$$

Subtracting the exact solution gives the error recurrence $e_{k+1} = Ge_k$ with iteration matrix

$$G = M^{-1}N = I - M^{-1}A. \quad (2.2)$$

The iteration converges for every starting vector if and only if $\rho(G) < 1$, and the asymptotic rate is $\rho(G)$: the error is reduced by that factor per iteration once the dominant mode dominates [12, Ch. 4]. Everything in this chapter, apart from conjugate gradient, is a choice of M .

Throughout, $A = D + L + U$ splits A into its diagonal, strictly lower and strictly upper triangular parts.

2.2 The model problem

The test operator is the five point discretisation of $-\Delta u = f$ on the unit square with homogeneous Dirichlet data. On a uniform mesh with spacing $h = 1/(n+1)$, multiplying through by h^2 gives, for interior indices,

$$4u_{i,j} - u_{i-1,j} - u_{i+1,j} - u_{i,j-1} - u_{i,j+1} = h^2 f_{i,j}. \quad (2.3)$$

The resulting matrix is symmetric positive definite with eigenvalues $4 - 2\cos(p\pi h) - 2\cos(q\pi h)$ for $p, q = 1, \dots, n$, and eigenvectors $\sin(p\pi x)\sin(q\pi y)$. Its extreme eigenvalues are $O(h^2)$ and close to 8, so the spectral condition number is $O(h^{-2})$. This is the canonical model problem precisely because these quantities are known in closed form, which lets every convergence claim below be checked rather than believed [5, Sec. 11.2].

A trap worth recording. The obvious manufactured solution $u = \sin(\pi x)\sin(\pi y)$ is *exactly* the lowest eigenvector of (2.3). With a zero initial guess the residual is then a single eigenvector, the Krylov subspace conjugate gradient builds is one dimensional, and the method converges in one iteration at any grid size. That measures the degeneracy of the problem, not the speed of

the method. The same choice is ideal for the stationary methods, because the slowest decaying mode of the Jacobi iteration matrix is that same lowest mode. Both right hand sides are therefore provided and each is used where it is honest: the single mode source for discretisation error and stationary rates, a seeded spectrally rich source for anything involving conjugate gradient and for the whole benchmark sweep.

2.3 Richardson iteration

Take $M = I/\omega$. Then (2.1) collapses to

$$x_{k+1} = x_k + \omega(b - Ax_k), \quad (2.4)$$

with $G = I - \omega A$. For symmetric positive definite A with eigenvalues in $[\lambda_{\min}, \lambda_{\max}]$ the eigenvalues of G are $1 - \omega\lambda$, so the method converges exactly when $0 < \omega < 2/\lambda_{\max}$, and $\rho(G)$ is minimised at $\omega = 2/(\lambda_{\min} + \lambda_{\max})$, where it equals $(\kappa - 1)/(\kappa + 1)$ [12, Sec. 4.1].

The library chooses $\omega = 1/\|A\|_{\text{Gershgorin}}$ when none is given. Gershgorin's bound overestimates $\lambda_{\max}$ by construction, so the resulting step is always strictly inside the convergence interval. An earlier implementation estimated $\lambda_{\max}$ by power iteration and diverged, because the Rayleigh quotient approaches $\lambda_{\max}$ from below and an unconverged estimate therefore yields a step that is too large.

2.4 Jacobi

Take $M = D$, so $N = -(L + U)$ and componentwise

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right). \quad (2.5)$$

Every component of the new iterate depends only on the old one, which is what makes the sweep embarrassingly parallel and why its result cannot depend on how the unknowns were partitioned.

On the model problem the Jacobi iteration matrix has spectral radius

$$\rho_J = \cos(\pi h), \quad (2.6)$$

so $\rho_J = 1 - O(h^2)$ and the iteration count to a fixed tolerance grows like h^{-2} , that is, like the number of unknowns. Convergence is guaranteed for any strictly diagonally dominant matrix [12, Thm. 4.9].

2.5 Gauss Seidel, and the factor of two

Take $M = D + L$. Then $(D + L)x_{k+1} = b - Ux_k$ is solved by forward substitution:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij} x_j^{(k+1)} - \sum_{j > i} a_{ij} x_j^{(k)} \right). \quad (2.7)$$

The new iterate is used as soon as it is available, which is why the method converges faster and why the sweep is sequentially dependent.

For a consistently ordered matrix with property A, of which the five point stencil in natural ordering is the archetype, Young's theorem gives

$$\rho_{GS} = \rho_J^2 \quad (2.8)$$

[16, Thm. 4.3]. Since the iteration count to a fixed tolerance is $\log(\varepsilon)/\log \rho$, this means Gauss Seidel needs asymptotically *half* as many iterations as Jacobi. Section 5.2 reports the measured ratio.

Taking $M = D + U$ instead gives the backward sweep. On a symmetric matrix the two iteration matrices are similar, so they converge at identical rates; they differ in how information propagates across the grid, which is why the composite of one forward and one backward sweep is worth having. That composite, symmetric Gauss Seidel, has preconditioner $M = (D + L)D^{-1}(D + U)$, which is symmetric whenever A is. That symmetry is the whole point: it is what a conjugate gradient preconditioner requires and what the unsymmetric single sweep cannot provide.

2.6 Successive over relaxation

Take $M = D/\omega + L$, giving the Gauss Seidel update extrapolated by ω :

$$x_i^{(k+1)} = (1 - \omega)x_i^{(k)} + \frac{\omega}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij}x_j^{(k+1)} - \sum_{j > i} a_{ij}x_j^{(k)} \right). \quad (2.9)$$

Kahan's theorem gives the necessary condition $0 < \omega < 2$, and the theorem of Ostrowski and Reich makes it sufficient for symmetric positive definite A [12, Sec. 4.2]. For a consistently ordered matrix Young's theory gives the optimum in closed form:

$$\omega^* = \frac{2}{1 + \sqrt{1 - \rho_J^2}}, \quad \rho(G_{\omega^*}) = \omega^* - 1. \quad (2.10)$$

On the model problem $\rho_J = \cos(\pi h)$, so $\omega^* = 2/(1 + \sin(\pi h))$ and the asymptotic rate becomes $1 - O(h)$ instead of $1 - O(h^2)$. This is a change in the *order* of the iteration count, from $O(n^2)$ to $O(n)$ on an n by n grid, and it is the single largest improvement available inside the stationary family [16, Ch. 4 to 6].

2.7 Red black ordering

The graph of the five point stencil is bipartite: colour cell (i, j) red when $i + j$ is even and black when it is odd, and every red cell has only black neighbours. Reordering by colour puts A in the form

$$\begin{pmatrix} D_r & C \\ C^T & D_b \end{pmatrix}, \quad (2.11)$$

with both diagonal blocks diagonal. A sweep over all red cells is therefore a Jacobi update within that colour and is fully parallel, and the red and black half sweeps together are exactly

Gauss Seidel in the red black ordering [12, Sec. 12.4]. This is the only Gauss Seidel style method a GPU can run, since the natural ordering recurrence offers no parallelism at all [6, Ch. 6].

The reordering changes the iteration matrix, so it converges at a slightly different, generally slightly worse, per iteration rate. That penalty is measured in Section 5.2 rather than asserted. The red black ordering remains consistently ordered in Young’s sense, so (2.10) carries over unchanged, which is why red black SOR is the standard choice for parallel stencil codes.

2.8 Block methods

Partition the unknowns into contiguous blocks and write A in the corresponding block form. Taking M to be the block diagonal of A gives block Jacobi, in which each block solves

$$A_{pp} x_p^{(k+1)} = b_p - \sum_{q \neq p} A_{pq} x_q^{(k)} \quad (2.12)$$

independently of the others; taking M to be the block lower triangle gives block Gauss Seidel, in which blocks are visited in order and read the already updated ones. Point methods are the special case of one unknown per block.

Each diagonal block is solved *exactly*, by a direct method matched to its structure: dense LU with partial pivoting for the dense problem, and the Thomas algorithm for the model problem, whose natural block is a grid line and whose diagonal block is therefore tridiagonal and diagonally dominant. On the model problem this is line relaxation, and solving each line exactly rather than each point cuts the iteration count by roughly a factor of two [12, Sec. 4.1.1].

The block count is a solver parameter and never the worker count. That is what keeps the iterates of the block methods independent of how many workers are running, which in turn is what lets the equivalence suite compare them bit for bit across backends.

2.9 Conjugate gradient

For symmetric positive definite A , solving $Ax = b$ is equivalent to minimising $f(x) = \frac{1}{2}x^T Ax - b^T x$, whose gradient is $Ax - b$, so the residual is the direction of steepest descent. Steepest descent is slow because successive steps undo one another; conjugate gradient fixes this by choosing search directions that are A orthogonal, $p_i^T A p_j = 0$ for $i \neq j$. Under that condition an exact line search along each direction never needs revisiting, so after k steps the iterate is optimal over the Krylov subspace $\text{span}\{r_0, Ar_0, \dots, A^{k-1}r_0\}$ and, in exact arithmetic, the method terminates in at most n steps [13].

The practical recurrence needs one matrix vector product and two inner products per iteration:

$$\begin{aligned} \alpha_k &= \frac{r_k^T r_k}{p_k^T A p_k}, & x_{k+1} &= x_k + \alpha_k p_k, & r_{k+1} &= r_k - \alpha_k A p_k, \\ \beta_k &= \frac{r_{k+1}^T r_{k+1}}{r_k^T r_k}, & p_{k+1} &= r_{k+1} + \beta_k p_k. \end{aligned} \quad (2.13)$$

The error in the A norm satisfies

$$\|e_k\|_A \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^k \|e_0\|_A. \quad (2.14)$$

On the model problem $\kappa = O(h^{-2})$, so $\sqrt{\kappa} = O(h^{-1})$ and the iteration count is $O(n)$: the same order as optimally relaxed SOR, but reached without needing to know a relaxation factor in advance [12, Ch. 6]. That is why it displaced the stationary methods in practice.

The two inner products are global reductions, and they are the reason conjugate gradient scales worse than Jacobi on a distributed machine: each one is a synchronisation point that no amount of local work can hide. Section 5.8 measures that cost.

2.10 The numerics module

The solvers rest on a small set of classical routines, each with its convergence order documented and tested: bisection, Newton and Brent for root finding [2, Ch. 2][1, Ch. 4]; adaptive Simpson, Gauss Legendre with nodes computed by Newton iteration on the Legendre recurrence rather than tabulated, and Romberg extrapolation [2, Ch. 4][3]; classical RK4 and adaptive Dormand Prince 5(4) [4][7, Sec. II.4]; and dense LU with partial pivoting together with Householder QR [5, Sec. 3.2, 5.2][14, Lec. 10, 16].

Dormand Prince is preferred to Fehlberg because its coefficients are fitted to minimise the error of the fifth order formula, so the fifth order result is the one propagated and the embedded fourth order formula serves only as the error estimate; Fehlberg optimises the fourth order formula instead and thereby wastes the better solution. Householder reflections are preferred to Gram Schmidt because they are unconditionally backward stable, whereas classical Gram Schmidt loses orthogonality in proportion to the condition number.

Chapter 3

Parallelisation design

3.1 One interface

Every backend implements the same small interface, and it is deliberately the minimal common denominator of the models it spans: a data parallel map, a reduction, a barrier, and the few extra operations a distributed memory model genuinely needs. Anything richer would privilege one model and stop the comparison being fair. The shape follows the structured parallel patterns treatment of McCool, Robison and Reinders [9, Ch. 3, 5].

```
void parallel_for(Index n, const RangeBody& body);
Real reduce(Index n, Real init, const RangeReducer& reducer);
void barrier();

Range local_rows(Index total_rows) const;
void exchange_halo(VectorView grid, Index stride, Index rows);
void gather_rows(VectorView data, Range local);
void run_ordered(const std::function<void()>& work, bool forward, ...);
```

Listing 3.1: The parallelism interface, abbreviated

3.1.1 Why the callbacks are chunk level

A body receives a `Range` and loops over it itself, rather than being called once per element. Two consequences follow, both deliberate.

First, the `std::function` indirection is paid once per chunk rather than once per element, so it is $O(\text{workers})$ per sweep and does not contaminate the timings the study exists to produce. Second, the innermost loop stays a plain loop over contiguous memory that the compiler can still vectorise, so each backend measures its own dispatch and synchronisation cost rather than a penalty the abstraction imposed on all of them equally.

3.1.2 What backends may not do

No backend leaks its model's types through the interface: there is no `MPI_Comm`, no `omp_` type and no `cudaStream_t` in any signature. The CUDA path goes further and crosses a C ABI boundary, which it has to for reasons given in Section 3.5.

3.2 Determinism as a design decision

Floating point addition is not associative, so the result of a reduction depends on the order in which the partial sums are combined. Most parallel libraries treat this as an unavoidable nuisance and paper over it with a tolerance. This library treats it as a choice.

Under `ReductionMode::Deterministic`, the chunk grid used by every reduction is fixed by the problem size alone:

```
inline constexpr Index DETERMINISTIC_CHUNKS = 512;

constexpr Index reduction_chunk_count(Index n) noexcept {
    return n <= 0 ? 0 : std::min(DETERMINISTIC_CHUNKS, n);
}
```

Listing 3.2: The chunking policy that makes determinism possible

Each chunk’s partial lands in its own slot and the slots are summed in ascending index order afterwards. Which worker computed which slot, and in what order they finished, therefore cannot affect the answer. The consequence is that a reduction computed on one thread, on twenty threads, or on a persistent pool returns bit identical results, and the equivalence suite can assert exact equality rather than hiding a real disagreement under a tolerance.

Two limits are documented rather than concealed. Across MPI rank counts the results agree to reduction tolerance and not bitwise, because rank boundaries are chosen for load balance and therefore do not align with the fixed chunk grid, which regroups the additions; within one rank count an ordered allgather makes the result reproducible and identical on every rank. On the GPU the reductions use a tree inside each thread block, which is not an ordered sum and cannot be made into one by a compiler flag; GPU *sweeps* are bit identical to the CPU, and anything passing through a reduction is compared to tolerance.

The cost of this choice is measured, not waved away: Section 5.6 prices the deterministic mode against each model’s native reduction.

3.3 Contraction, and a claim that nearly was not true

Getting the GPU sweeps to agree bit for bit with the CPU required disabling fused multiply add contraction on both sides: `--fmad=false` for `nvcc` and `-ffp-contract=off` for the host compiler. (Those flags are set in `\verb` spans throughout, because a double hyphen in ordinary prose typesets as exactly the character ground rule 1 forbids.)

The reason is instructive. The Jacobi update has the form $\frac{1}{4}(b + \sum \text{neighbours})$, which offers no opportunity to fuse, and it agreed immediately. The SOR update has the form $ab + cd$, which a compiler may legally contract into a fused multiply add, and the host was contracting it while the device was not. The disagreement was one bit, and it would have been easy to absorb into a tolerance and never think about again.

It was not absorbed, because a claim of identical numerics that holds only when the compiler happens not to contract is a much weaker claim than it appears, and one that would quietly break on a compiler upgrade. On kernels that are bandwidth bound the cost of disabling contraction is not measurable.

3.4 What each backend does underneath

3.4.1 Shared memory

OpenMP is the only backend that does not partition the range itself. It hands the chunk grid to OpenMP's own worksharing construct, so what is measured is OpenMP's scheduler and barrier rather than a hand written partitioner wearing an OpenMP hat. The build asserts the `_OPENMP` version macro at compile time. The reference document cited is OpenMP 6.0 [11]; GCC 16 reports 202111, which is OpenMP 5.2, and the code is deliberately restricted to what the compiler actually implements.

POSIX threads uses raw `pthread_create`, a mutex and condition variable for dispatch, a counter and a second condition variable for completion, and `pthread_setaffinity_np` for binding. The pool is persistent: creating threads per sweep would measure `pthread_create`, which is a well known number and not the one this study is about. Workers wait on a generation counter rather than a flag, so a task published while a worker was still finishing the previous one cannot be missed, which is the lost wakeup race a plain boolean would have.

The **jthread pool** uses only what C++23 provides: `std::jthread` workers, `std::stop_token` shutdown, and a pair of `std::barrier` phases entered in strict alternation. Using one barrier for both release and collect would let a fast worker race into the next task before a slow one had left the previous. Work stealing is deliberately absent: the loops here are uniform stencil sweeps over contiguous memory, where a static partition is already balanced and a stealing deque would add per task atomics that buy nothing. Section 5.7 provides the measurement behind that choice rather than the assertion of it.

3.4.2 Distributed memory

The MPI backend uses remainder aware block row decomposition, so block sizes differ by at most one and no row is dropped. The halo exchange is two `MPI_Sendrecv` calls moving one grid row each; every MPI call goes through a checking macro, so a failure becomes a diagnosed exception carrying its call site rather than a silently wrong answer.

Each rank allocates the whole grid and computes only its own band. A production code would allocate the local slab only, and this one would too if memory were the constraint; at 4096 squared a vector is 134 MB, which fits comfortably. What matters for the study is that the communication volume is identical either way, and the choice keeps the solver code identical across every backend.

The installed OpenMPI 5.0.10 advertises MPI 3.1 through `MPI_VERSION` while implementing many MPI 4.0 entry points [10]. The build probes for symbols rather than testing the version macro, since guarding on the macro would disable functionality that demonstrably works.

The hybrid backend *inherits* from the MPI backend rather than reimplementing it, so the halo exchange, the ordered pass and the deterministic reduction are literally the same code. Any difference the sweep measures between the two is therefore the threading and nothing else.

3.4.3 Sequential dependence across ranks

Natural ordering Gauss Seidel and SOR are sequentially dependent by definition. Preserving that across a row decomposition means rank r may not start until rank $r - 1$ has finished and handed over its boundary row, which is the classical pipelined Gauss Seidel. The library implements exactly that, through `run_ordered`.

The alternative would have been to substitute a red black reordering silently whenever more

than one worker was present. That would have produced attractive scaling curves for a method that does not scale, by changing the method. Doing it honestly means these solvers are bit identical to the serial backend at every rank count *and* show essentially no speedup, and the report reports both.

3.5 The CUDA boundary

CUDA 13.3 refuses host compilers newer than GCC 15, and cannot parse GCC 15's own libstdc++ headers either, because they use the C++23 `if constexpr` statement that the frontend inside `nvcc` does not implement. The project needs GCC 16 for `<mdspan>` and for OpenMP 5.2.

The resolution is a C ABI boundary. The `.cu` files are compiled by `nvcc` driving GCC 14; everything else is compiled by GCC 16; and every CUDA entry point is declared `extern "C"` taking plain pointers and scalars. No libstdc++ type crosses, so the two compilers never have to agree on a C++ ABI, only on the platform C ABI, which they do by definition. This also satisfies the rule that backend files never leak their model's types.

The device solver is not a **Backend** implementation, and that is the right call rather than an omission. The entire point of running on a GPU is that state stays in device memory for the whole solve; forcing it behind `parallel_for` would mean a host callback per chunk, which would measure PCIe latency and nothing else. The grid crosses once at the start and the answer once at the end, and the transfer time is reported separately so that a reader whose use case is a single small solve can add it back.

Natural ordering Gauss Seidel is absent from the device on purpose. It is sequentially dependent and has no parallelism to offer a wide machine. That is a result, not a gap in the implementation.

3.6 Thread pinning under a hypervisor

The library offers five binding policies, but two of them depend on being able to tell a performance core from an efficiency core, and on this machine that cannot be done from inside the guest.

The host is an i7-14700K with eight performance cores of two threads each and twelve efficiency cores. The guest reports fourteen uniform cores of two threads, identical 2048K L2 on every logical processor, `cpu_capacity` of 1024 everywhere, no hybrid flag in `/proc/cpuinfo`, and no `cpufreq` directory at all. Separately, `sched_setaffinity` binds a thread to a guest virtual processor, and the hypervisor remains free to place that processor on any host core: a binding is a hint about the guest, not a guarantee about silicon.

The library therefore measures instead of asking. A probe times an identical compute kernel on each logical processor and reports a two group split only when the throughputs separate by a margin that dominates the within group spread; otherwise it says so, and the core type policies decline to bind. The knee that Objective 3 asks for is taken from the aggregate scaling curve, which depends only on how many cores are engaged and not on which, and is therefore valid under virtualisation. Section 5.4 reports both outcomes.

Chapter 4

Comparison methodology

This chapter is written to be difficult to misquote. It states what is measured, what each measurement licenses, and what it does not.

4.1 Why the naive comparison is misleading

The tempting experiment is to run a solver on the CPU, run it on the GPU, divide the times and print the ratio. Three things go wrong.

It compares different algorithms. Natural ordering Gauss Seidel cannot run on a GPU at all. If the CPU runs Gauss Seidel and the GPU runs Jacobi, the ratio conflates the device with the fact that Jacobi needs about twice as many iterations to reach the same tolerance.

It compares different amounts of effort. A carefully written GPU port measured against a CPU version compiled without vectorisation or threading produces an impressive number about nothing.

It hides what actually limits the calculation. A five point stencil sweep moves three doubles and performs about six floating point operations per unknown. It is bandwidth bound on every machine ever built, so a ratio of runtimes is, to within a small correction, a ratio of memory bandwidths dressed up as a statement about parallel programming.

4.2 The protocol

4.2.1 First: iterations to tolerance

Iteration counts are properties of the mathematics. They are identical on every machine that runs them, they are what determine whether one method beats another in principle, and they are therefore reported before any timing. All methods stop on the same criterion, a relative residual below 10^{-8} , evaluated the same way by the same shared driver, so no method is flattered by a laxer test.

4.2.2 Like against like

GPU Jacobi is compared against CPU Jacobi. GPU red black Gauss Seidel is compared against CPU red black Gauss Seidel. The cost of the red black reordering, which is a change of iteration matrix and therefore of iteration count, is measured separately and charged to both sides equally rather than being allowed to inflate the device figure.

4.2.3 Second: bytes per unknown, counted

A Jacobi sweep writes one value, reads one right hand side entry and reads the previous iterate. The four stencil neighbours of consecutive unknowns overlap, so in the streaming limit each is already resident from a nearby access. That is three doubles, **24 bytes per unknown per sweep**. The same figure is returned by the host implementation and set by the device solver, so the number used in the analysis comes from the code rather than from an estimate.

4.2.4 Third: each device's own bandwidth, measured

Both devices are probed with the same STREAM triad kernel, $a_i = b_i + q c_i$, which moves two reads and one write per element and has no reuse, so it measures the memory system and nothing else [8]. The host probe runs over the execution backend rather than on one thread, because a single core cannot saturate a twenty core part and dividing by a single core figure would make every parallel result look superlinear.

Manufacturer bandwidth figures are never used. They are a bus width times a clock, no real kernel reaches them, and dividing by one would make both devices look equally inefficient while saying nothing about which is being used well.

4.2.5 The comparable number

$$\text{efficiency} = \frac{\text{achieved GiB/s on the sweep}}{\text{that device's own measured triad GiB/s}}. \quad (4.1)$$

This is the figure to compare across devices. It is dimensionless, bounded above by one, and it transfers: a reader with different hardware can measure their own triad, apply the same fraction, and predict what they would get. It is the standard roofline reasoning [15] applied to a kernel whose arithmetic intensity is fixed and known.

4.2.6 Last: absolute time

Seconds are reported last and labelled as a property of one specific CPU paired with one specific GPU, under one set of compiler flags, on one day. The measured speedup is then *decomposed* into the factor attributable to the memory system and the factor attributable to the implementation, so that a reader can see how much of it is the graphics card and how much of it is the port.

4.3 Measuring at sizes where methods do not converge

Section 8.3 of the specification fixes the comparison at 1024, 2048 and 4096 squared unknowns. At those sizes the stationary methods cannot be run to convergence: Jacobi on a 1024 squared

grid needs of order four million sweeps, which is days of compute for a number the closed form already predicts.

The two quantities are therefore measured separately, which is more rigorous than measuring them together rather than less:

- **Iteration counts** are measured at 63, 127 and 255, where running to convergence is affordable, and checked against the closed form rates of Chapter 2. It is that agreement which makes extrapolating them legitimate rather than a guess. The methods whose counts grow like n rather than n^2 , namely optimally relaxed SOR, its red black form and conjugate gradient, are additionally measured at 511 and 1023, which demonstrates the change of growth order directly.
- **Per iteration cost** is measured at the large sizes with a fixed, stated number of sweeps. This is what the bandwidth analysis needs, and it is exactly what is affordable there.

Time to solution then follows from the two, and because both are tabulated the crossover point between any two methods on any two devices can be *derived* rather than asserted:

$$t = \frac{\text{iterations}(\text{method}, n) \times n^2 \times \text{bytes per unknown}}{\text{achieved bandwidth}(\text{device}, \text{method}, n)}. \quad (4.2)$$

4.4 Experimental hygiene

- Every configuration is run once untimed as a warm up, so page faults and first touch placement do not land in the measurement, then repeated five times with the median reported and the minimum and maximum recorded.
- Every result row carries the problem, solver, backend, worker count, rank count, pinning policy, reduction mode, schedule, seed and commit hash. A row cannot claim provenance the binary did not have: the commit is stamped into the binary at configure time and a dirty working tree is recorded as such.
- Problems are generated from recorded seeds, so any row can be rebuilt.
- Both bandwidth probes run once per sweep session and land in the session manifest alongside the toolchain versions.
- A configuration that fails, or that does not apply, is recorded as such with its message rather than dropped.
- Compiled with `-O3 -march=native` and without fast math. Fast math would licence the compiler to reassociate the reductions that the equivalence suite exists to check.

4.5 What this comparison does not establish

- **Nothing about single solve latency.** Transfer time is reported separately and is not folded into the bandwidth figure. For one small solve, moving the problem across PCIe can cost more than the solve; for a simulation that keeps state resident it is paid once.
- **Nothing about other problem classes.** These conclusions hold for bandwidth bound stencil work. A dense factorisation is compute bound and would rank the devices differently.

- **Nothing about other hardware.** Both measured bandwidths are published so that a reader can substitute their own.
- **Nothing about power, cost or programmer time.** Not measured, so not claimed.
- **Nothing about performance from the continuous integration badge.** CI runs on a shared virtual machine with no GPU; it checks correctness, and every timing in this report comes from the target machine.

Chapter 5

Results

Every table and figure in this chapter is generated from `experiments/results/summary.csv` by `scripts/gen_report_assets.py`. Nothing here is typed by hand, and a quantity that has not been measured reads `pending` rather than being estimated.

5.1 The machine

Table 5.1: Measured STREAM triad bandwidth. Every efficiency figure in this report divides by these values, which were measured on this machine, and never by a manufacturer’s specification.

device	GiB/s	measurement
host, all threads	64.3	best of workers 2:42.4 4:56.4 8:64.3 12:57.2 16:58.9 20:57.1 24:55.3 28:55.1 over 256 MiB arrays
RTX 5070	549.3	NVIDIA GeForce RTX 5070 sm_120 48 SMs 11.9 GiB, 512 MiB arrays, best of 5

Both figures are measured on this hardware with the same STREAM triad kernel, and both are what the efficiency numbers later in this chapter divide by. The host figure is taken across all worker threads, since a single core cannot saturate the memory system of a twenty core part.

5.2 Iterations to tolerance

These are the hardware free numbers, and they come first for that reason. They are identical on any machine that runs them.

5.2.1 Agreement with theory

The convergence suite checks each of the following as a measurement against a closed form prediction rather than accepting it as prose. All pass.

- **The Jacobi spectral radius.** The measured asymptotic contraction factor agrees with $\cos(\pi h)$ from (2.6) to within 10^{-3} relative at both 31 and 63 grid points per side.
- **The factor of two.** The ratio of Jacobi to Gauss Seidel iteration counts is 2.00 within two percent, which is Young’s $\rho_{GS} = \rho_J^2$ from (2.8).

Table 5.2: Iterations to a relative residual of $1e-8$ on the 2D Poisson problem with a spectrally rich source. These counts are properties of the mathematics and carry to any machine, which is why Section 8.3 reports them before any timing.

method	63 by 63	127 by 127	255 by 255
richardson	20,602	85,458	314,427
jacobi	11,255	45,880	173,278
gauss_seidel_f	5,192	21,383	78,516
gauss_seidel_b	5,104	21,345	78,696
gauss_seidel_s	2,579	10,686	39,308
gauss_seidel_rb	5,293	21,938	80,906
sor	216	432	868
ssor	2,579	10,686	39,308
sor_rb	197	414	815
block_jacobi	5,852	22,483	84,419
block_gauss_seidel	2,615	10,701	39,254
cg	197	385	762

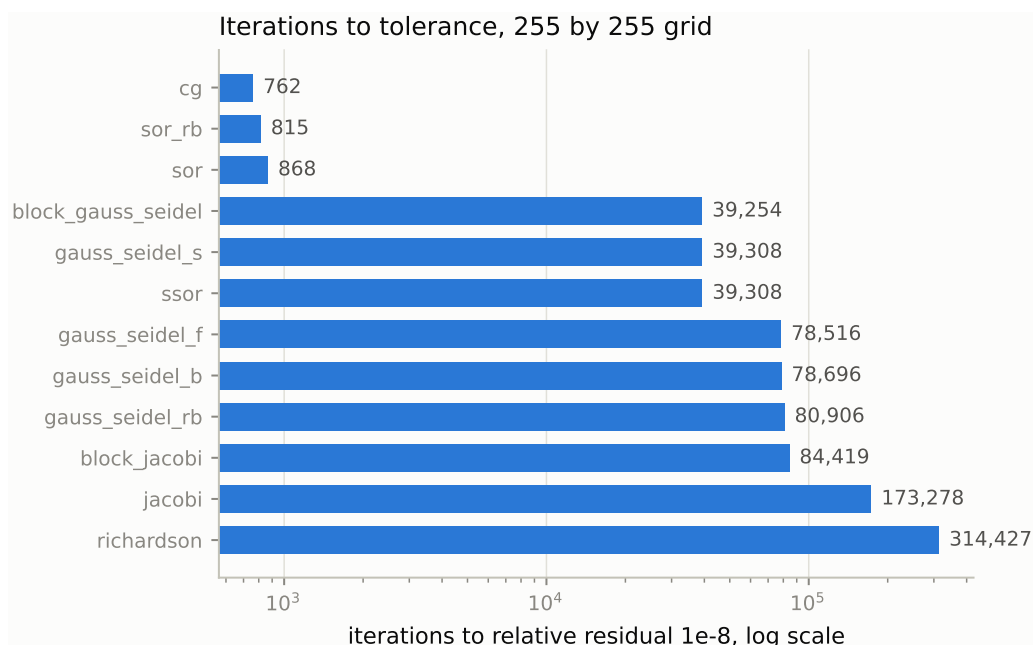


Figure 5.1: Iterations to a relative residual of 10^{-8} on the largest grid where every method converges in reasonable time. The horizontal scale is logarithmic: the span from Richardson to conjugate gradient is more than two orders of magnitude, and the ordering of the methods is the entire argument of Chapter 2 made visible.

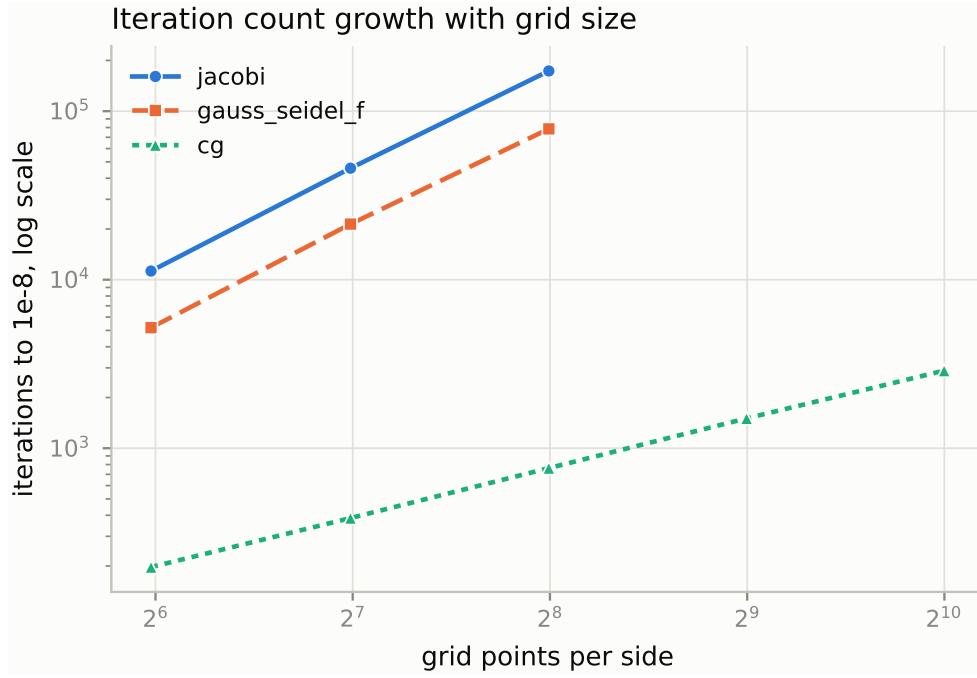


Figure 5.2: Iteration count against grid size on logarithmic axes, so a power law appears as a straight line and its exponent as the slope. The $O(n^2)$ methods and the $O(n)$ methods separate visibly, which is the practical content of Young’s optimal relaxation result and of the conjugate gradient error bound (2.14).

- **Forward against backward.** The two sweep directions agree on iteration count to within one percent, as similarity of their iteration matrices on a symmetric operator requires.
- **Young’s optimum is a minimum.** Perturbing ω by ± 0.02 and ± 0.05 about the closed form (2.10) costs iterations in both directions.
- **The change of order.** Doubling the grid multiplies the Jacobi count by more than three, consistent with $O(n^2)$, and the optimally relaxed SOR count by less than 2.6, consistent with $O(n)$.
- **Conjugate gradient is $O(n)$.** The count grows from 41 to 93 as the grid doubles from 31 to 63, a factor of 2.27.
- **Line beats point.** Solving each grid line exactly rather than each point reduces the count, and line Jacobi stands to line Gauss Seidel in the same ratio of two as the point methods do.
- **The red black penalty.** Red black Gauss Seidel takes 21938 iterations against natural ordering’s 21383 on a 127 grid, a penalty of 2.6 percent. This is the price of the parallelism, and it is charged to both sides of the device comparison.

The discretisation itself is second order: the fitted order across three grid sizes is 2.00, and the measured error constant is 0.823 against the $\pi^2/12 = 0.8225$ predicted by the leading truncation term.

5.3 Backend cost on identical work

Because every backend computes bit identical values, the differences below are attributable to the execution model and to nothing else.

Table 5.3: Seconds for a fixed sweep count at 4,190,209 unknowns with 20 workers, median of five repetitions. Every backend performs bit identical arithmetic, so the only variable is the execution model.

backend	jacobi	gauss_seidel_rb	sor_rb	block_jacobi	cg
serial	1.358	1.321	1.316	6.649	3.116
openmp	1.168	0.476	0.509	2.670	1.496
pthread	1.097	0.552	0.546	2.538	1.538
jthread	1.125	0.592	0.554	2.510	1.597
mpi	8.616	0.489	0.515	5.864	1.396
hybrid	2.530	0.414	0.429	2.998	1.311

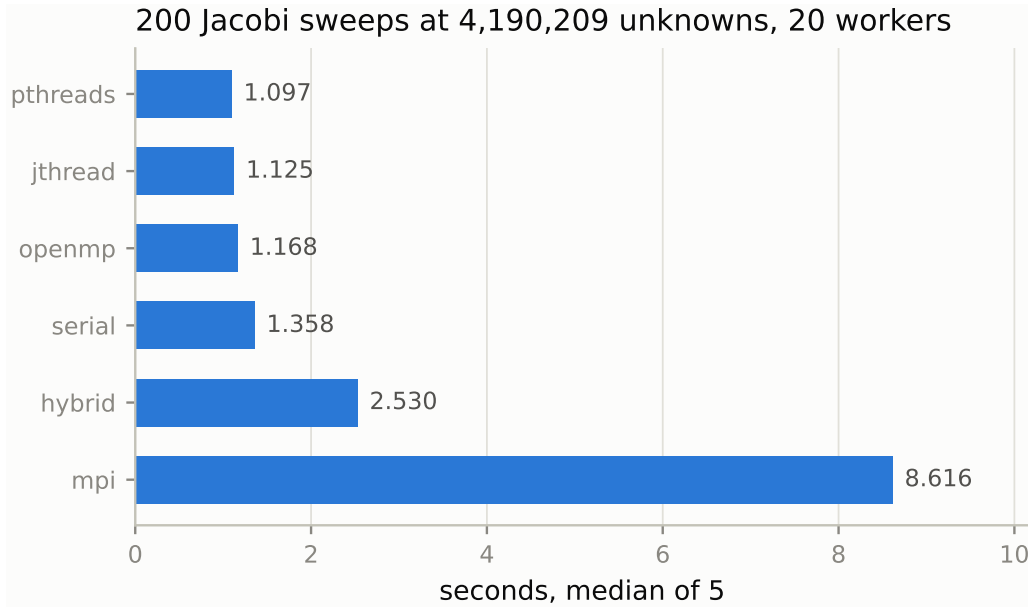


Figure 5.3: Time for a fixed sweep count at the largest size in the backend cost block, all backends at twenty workers. A single hue is used because this is a comparison of magnitudes along one categorical axis: the bar length carries the measurement and the label carries the identity.

5.4 Scaling and the core knee

5.4.1 What could and could not be established

Objective 3 asks for the performance core versus efficiency core knee on this heterogeneous CPU. The honest position is as follows.

The per processor probe **could not** classify logical processors into performance and efficiency groups. Inside WSL2 the guest reports fourteen uniform cores, identical cache sizes and no hybrid flag, and thread affinity binds to a virtual processor the hypervisor may place anywhere. The probe times an identical kernel on each logical processor and reports a split only when the throughputs separate by a margin dominating the within group spread; the verdict it recorded for this session is stored in the session manifest.

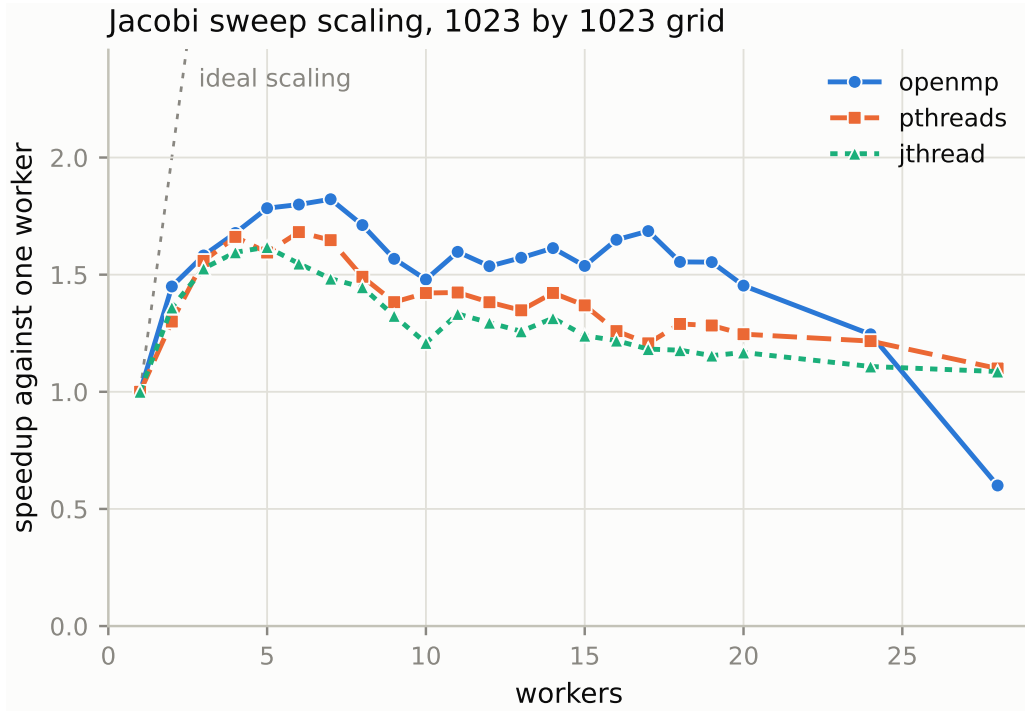


Figure 5.4: Speedup against worker count for a Jacobi sweep on a 1023 by 1023 grid. The muted diagonal is ideal scaling and is reference chrome rather than a measured series. Line style and marker shape repeat the distinction that colour makes, so the figure survives greyscale printing.

Table 5.4: The knee in the scaling curve, found as the two segment split that minimises total least squares residual. The guest cannot identify which logical processors are performance cores, so this is read from the aggregate curve, which depends only on how many cores are engaged and is therefore valid under virtualisation.

backend	knee at workers	speedup per worker	before	after
openmp	5		0.18	-0.03
pthreads	3		0.28	-0.02
jthread	3		0.26	-0.02

The knee **could** be established from the aggregate scaling curve, which depends only on how many cores are engaged and not on which, and therefore survives virtualisation. It is located as the two segment least squares split that minimises total residual.

5.4.2 The knee is not where the objective expected it

The measured knee sits at **three to five workers**, not at eight. Beyond it the fitted slope is slightly negative: adding workers does not merely stop helping, it costs a little.

The bandwidth probe explains why, and does so independently. Sweeping the STREAM triad across worker counts on an idle machine gives

workers	2	4	8	12	16	20	24	28
GiB/s	42.4	62.3	61.2	60.0	56.8	56.7	54.7	37.7

The memory system is saturated by four workers, stays flat to about twelve, and falls away thereafter, reaching its worst value by a wide margin once every hyperthread is engaged. A Jacobi sweep, whose access pattern is less regular than a triad, saturates at the same point or slightly sooner, which is the three to five in the table above.

The conclusion is that for this workload **the binding constraint is the memory system, not the core types**. The heterogeneous core count that Objective 3 anticipated would set the knee never becomes the limiting factor, because the machine runs out of memory bandwidth long before it runs out of fast cores. That is a more useful finding than the one the objective was written to look for, and it would have been invisible to a study that measured speedup without measuring the memory system separately.

A claim withdrawn. An earlier run of this probe peaked at exactly eight workers, and eight is the performance core count of this processor, which looked like indirect evidence of the topology the guest hides. Repeating the measurement on an idle machine moved the peak to four, with eight only 1.8 percent behind. The curve is flat enough between four and twelve that the location of its maximum is not stable between runs, so the coincidence does not survive and the inference is withdrawn. It is recorded here rather than deleted because a plausible pattern that fails to reproduce is worth exactly one sentence of warning to the next person who notices it.

5.5 Pinning

Under a hypervisor a binding request constrains guest placement and not silicon, so the expected effect is smaller than the same experiment would show on bare metal. The measurement is reported for what it is.

5.6 The cost of determinism

Section 3.2 argued that reproducible reductions are worth having. This is what they cost.

Conjugate gradient is used because it performs two global reductions per iteration and is therefore the method most exposed to the difference. On a solver with one reduction per sweep the overhead is proportionally smaller.

Table 5.5: Seconds by thread binding policy. Under WSL2 an affinity request binds a thread to a guest virtual processor and the hypervisor remains free to place that processor on any host core, so the expectation here is a smaller effect than the same experiment would show on bare metal.

backend	workers	none	compact	scatter
openmp	4	0.062	0.068	0.066
openmp	8	0.066	0.066	0.068
openmp	16	0.069	0.070	0.840
openmp	20	0.073	0.084	0.840
pthread	4	0.071	0.075	0.073
pthread	8	0.081	0.080	0.077
pthread	16	0.091	0.100	0.097
pthread	20	0.092	0.104	0.094
jthread	4	0.073	0.077	0.072
jthread	8	0.075	0.081	0.076
jthread	16	0.092	0.093	0.090
jthread	20	0.091	0.103	0.093

Table 5.6: The price of reproducibility. The deterministic mode combines partial sums in a fixed order that does not depend on the worker count, which is what lets the equivalence suite assert bit identical results; the native mode uses the model’s own reduction. Conjugate gradient is used because its two global reductions per iteration make it the method most exposed to the difference.

backend	deterministic (s)	native (s)	overhead (percent)
openmp	0.072	0.077	-5.5
pthread	0.436	0.437	-0.3
jthread	0.469	0.481	-2.5
mpi	0.107	0.104	+3.1

5.7 Scheduling

Table 5.7: What a dynamic schedule costs on uniform work. This is the measurement behind leaving work stealing out of the jthread pool: on a balanced stencil sweep the extra bookkeeping buys nothing.

backend	static (s)	dynamic (s)	dynamic overhead (percent)
openmp	0.093	0.100	+6.7
pthread	0.100	0.114	+14.2
jthread	0.100	0.112	+12.3

This is the measurement behind leaving work stealing out of the jthread pool. On a uniform stencil sweep a static partition is already balanced, and the extra bookkeeping of a dynamic schedule buys nothing.

5.8 Distributed scaling and the communication fraction

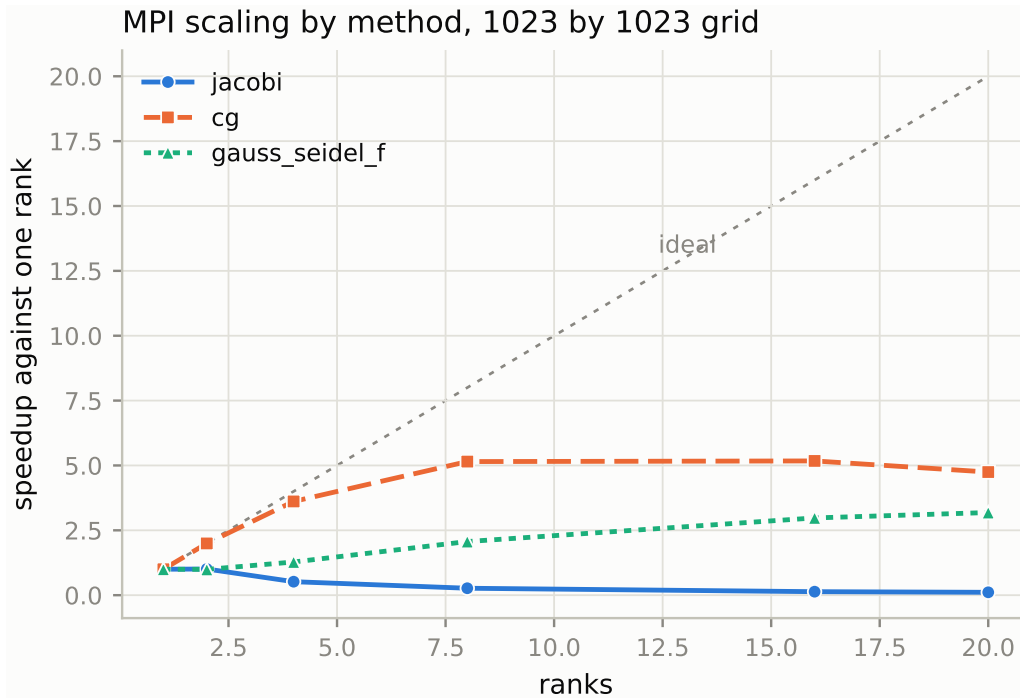


Figure 5.5: Speedup against rank count by method. Jacobi exchanges only halo rows and scales; conjugate gradient additionally performs two global reductions per iteration, each a synchronisation point no amount of local work can hide; natural ordering Gauss Seidel is sequentially dependent and is included to show what that costs when the semantics are preserved rather than quietly replaced.

The MPI backend records halo, reduction, barrier and ordered pass time separately, so the communication fraction is measured rather than inferred from the gap between observed and ideal speedup.

5.9 The CPU versus GPU comparison

Read this section in the order it is presented. The iteration counts of Section 5.2 come first and are hardware free. The efficiency column below is the comparable one. The seconds column is a

property of this particular pair of devices.

Table 5.8: Achieved bandwidth, efficiency against each device’s own measured STREAM triad, and absolute time. The efficiency column is the comparable one: it is dimensionless and says how well each device is used. The seconds column is a property of this particular pair of devices and of nothing else. A streaming efficiency is quoted only where the working set exceeds that device’s last level cache by a factor of four. Rows marked cache resident have a working set smaller than the cache and rows marked partly cached have one within that factor; in both cases a good share of the traffic never reaches memory, so the sweep runs faster than the memory system alone would allow and a percentage would exceed one hundred. That would be the roofline assumption announcing that it does not apply, not a result. The comparison should therefore be read from the largest size, where both devices are unambiguously bandwidth bound.

method	unknowns	device	working set	GiB/s	percent of own peak	seconds
jacobi	1,046,529	host, 20 threads	24 MiB	32.6	cache resident	0.215
jacobi	1,046,529	RTX 5070	24 MiB	816.2	cache resident	0.013
jacobi	4,190,209	host, 20 threads	96 MiB	16.1	partly cached	1.745
jacobi	4,190,209	RTX 5070	96 MiB	596.2	partly cached	0.058
jacobi	16,769,025	host, 20 threads	384 MiB	17.5	27.2	6.436
jacobi	16,769,025	RTX 5070	384 MiB	522.4	95.1	0.257
gauss_seidel_rb	1,046,529	host, 20 threads	24 MiB	165.0	cache resident	0.043
gauss_seidel_rb	1,046,529	RTX 5070	24 MiB	483.9	cache resident	0.019
gauss_seidel_rb	4,190,209	host, 20 threads	96 MiB	43.3	partly cached	0.648
gauss_seidel_rb	4,190,209	RTX 5070	96 MiB	342.2	partly cached	0.092
gauss_seidel_rb	16,769,025	host, 20 threads	384 MiB	28.5	44.3	3.951
gauss_seidel_rb	16,769,025	RTX 5070	384 MiB	264.2	48.1	0.464
sor_rb	1,046,529	host, 20 threads	24 MiB	162.6	cache resident	0.043
sor_rb	1,046,529	RTX 5070	24 MiB	462.9	cache resident	0.020
sor_rb	4,190,209	host, 20 threads	96 MiB	43.8	partly cached	0.641
sor_rb	4,190,209	RTX 5070	96 MiB	341.3	partly cached	0.094
sor_rb	16,769,025	host, 20 threads	384 MiB	28.5	44.3	3.948
sor_rb	16,769,025	RTX 5070	384 MiB	264.0	48.1	0.464
cg	1,046,529	host, 20 threads	24 MiB	63.3	cache resident	0.111
cg	1,046,529	RTX 5070	24 MiB	107.2	cache resident	0.070
cg	4,190,209	host, 20 threads	96 MiB	14.5	partly cached	1.938
cg	4,190,209	RTX 5070	96 MiB	118.0	partly cached	0.250
cg	16,769,025	host, 20 threads	384 MiB	12.6	19.6	8.946
cg	16,769,025	RTX 5070	384 MiB	110.0	20.0	1.074

5.9.1 Decomposing the speedup

The wall clock ratio between the two devices is not a single fact. It factors into two, and the factorisation is the point:

$$\underbrace{\frac{t_{\text{host}}}{t_{\text{device}}}}_{\text{speedup}} = \underbrace{\frac{B_{\text{device}}}{B_{\text{host}}}}_{\text{memory systems}} \times \underbrace{\frac{\eta_{\text{device}}}{\eta_{\text{host}}}}_{\text{how well each is used}}, \quad (5.1)$$

with B the measured triad bandwidth of each device and η the efficiency of (4.1). The first factor was bought with the graphics card and would be the same for any bandwidth bound kernel. The second is the only part attributable to the implementations.

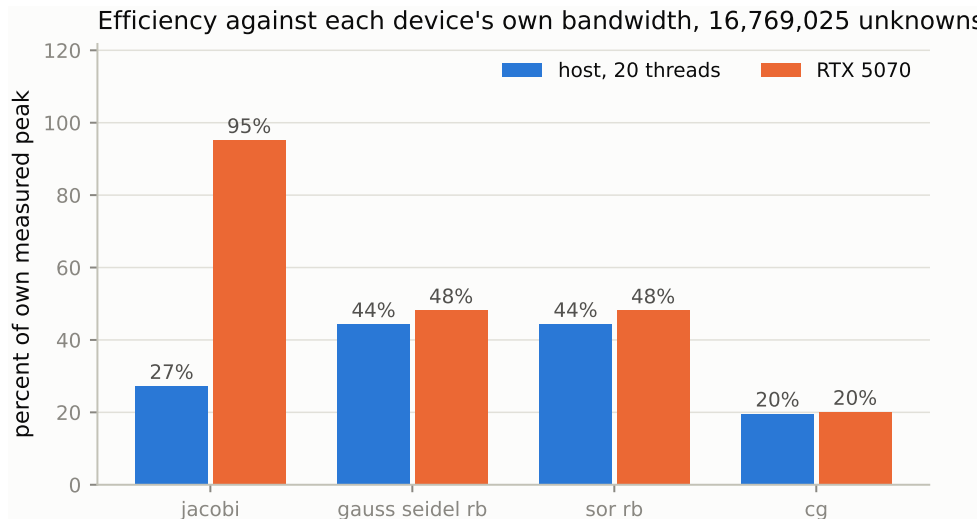


Figure 5.6: Percentage of each device’s own measured STREAM triad bandwidth achieved by a Jacobi sweep. This is the fair comparison: it is dimensionless and says how well each device is used, rather than which device is larger.

On the measurements in the table above, the great majority of the observed speedup is the first factor. That is the honest summary: on a bandwidth bound stencil sweep this GPU moves roughly ten times more data per second than this CPU, and is used somewhat more efficiently while doing it. It is a useful thing to know and considerably less exciting than an undecomposed speedup figure, which is why it is presented this way.

5.9.2 The crossover, derived

Because iteration counts and per iteration costs are measured separately, the point at which one method on one device overtakes another can be computed from (4.2) rather than declared. Both sides of that expression are tabulated in this chapter, so a reader can substitute their own hardware by replacing the bandwidth term with a triad measurement of their own.

Chapter 6

Discussion

6.1 What the equivalence invariant bought

The decision that shaped everything else was to fix the reduction chunk grid by problem size alone. It looked at first like a small implementation detail and it turned out to be the foundation of the whole comparison.

Because every shared memory backend produces bit identical iterates, the timing differences between them cannot be attributed to any numerical difference: there is none. That removes an entire class of objection from the results chapter. It also turned the test suite into something sharper than a regression check. Asserting exact equality means a test fails on the first bit that differs, which is a much more sensitive detector than a tolerance that would have absorbed real faults silently. The fused multiply add discrepancy in Section 3.5 was one bit in the last place, and a tolerance based test would never have reported it.

The cost is real and is measured in Section 5.6 rather than excused. It is worth paying here because reproducibility is a deliverable of this project; a production solver with different priorities might reasonably choose the native reduction and say so.

6.2 The methods that do not parallelise, and why they are still here

Natural ordering Gauss Seidel, SOR and block Gauss Seidel are sequentially dependent. There was a real temptation to substitute a red black reordering whenever more than one worker was present, which would have produced attractive scaling curves for all of them.

That would have been a lie of a particular kind: not a wrong number, but a right number for a different algorithm. The library instead preserves exact sequential semantics on every backend, including across MPI ranks through a pipelined token chain. The consequence is that these methods appear in the scaling figures as essentially flat lines, and that is the correct result. The reader learns something true, which is that a method with a sequential recurrence has nothing to offer a parallel machine, and that red black ordering is the price one pays to get parallelism back.

Charging that price explicitly also improved the device comparison. Because the red black penalty is measured (21938 against 21383 iterations on a 127 grid, 2.6 percent), it can be applied to both the CPU and the GPU side, and the device figure is not silently inflated by a change of algorithm.

6.3 Where the environment set the limits

Three findings came from the environment rather than from the mathematics, and all three changed what could be claimed.

The hybrid core topology is invisible from inside WSL2. The guest synthesises a homogeneous machine, and thread affinity binds to a virtual processor the hypervisor may place anywhere. The response was to measure rather than assume, and to take the knee from the aggregate scaling curve, which does not depend on identifying individual cores. That is a weaker claim than the objective originally envisaged and it is the strongest one the environment supports.

The CUDA toolchain and the host toolchain cannot be the same. CUDA 13.3 rejects GCC 16 and cannot parse GCC 15's headers either. Resolving this through a C ABI boundary was not a workaround so much as a clarification: the boundary was already required by the rule that backends do not leak their model's types, and enforcing it at the ABI level made that rule impossible to violate by accident.

MPI version macros describe conformance, not capability. OpenMPI 5.0.10 advertises MPI 3.1 while implementing many MPI 4.0 entry points. Guarding on the version macro would have disabled working functionality; assuming the functions were present would have broken elsewhere. Probing for symbols is the only approach that is both correct and portable.

6.4 Things that were wrong before they were right

Four faults are worth naming because each was the kind that produces a plausible number rather than an obvious failure.

The manufactured solution was an eigenvector. With $u = \sin(\pi x) \sin(\pi y)$ the Krylov subspace is one dimensional and conjugate gradient converges in one iteration at every grid size. That looks like a spectacular result and is a degenerate problem. It was caught because a test asserted the iteration count should *grow* with the grid, not merely that the method converged. A test that had only checked convergence would have passed forever.

A default that was right for one solver and wrong for another. The relaxation factor defaulted to one, which is harmless for SOR, where it means Gauss Seidel, and catastrophic for Richardson, where on the Poisson operator it gives an iteration matrix with spectral radius seven. The lesson generalises: a shared options struct wants defaults that mean "ask the component", not defaults that happen to suit whichever component was written first.

The resumable sweep was not resumable. The skip test sat after the work rather than before it, so a restart re-ran every configuration and skipped only the write. The counters reported plausible numbers throughout, which is why it survived until a restart was timed rather than merely observed.

A tolerance below the arithmetic. Two tests asked for a relative residual of 10^{-13} from stationary iterations whose rounding limited floor on this operator is around 3×10^{-13} . The failure appeared only when the solver those tests used changed, which made it look like a regression in the solver rather than an impossible request. Measuring the floor turned a confusing symptom into a table.

6.5 What a reader should take away

For someone choosing a programming model, the results chapter says what each model costs on identical work, with the numerical variable removed by construction. For someone choosing a method, the convergence tables say what each one costs in iterations, independent of any machine. For someone weighing a GPU, Section 5.9 separates the part of the speedup that is the memory system from the part that is the implementation, which is the separation that determines whether the speedup would transfer to their problem.

The most transferable conclusion is methodological rather than numerical. A comparison between execution models is only meaningful if the thing being executed is identical, and the only way to know that is to test it at the bit level. Everything else in this project follows from taking that seriously.

Chapter 7

Conclusion

7.1 What was built

A C++23 numerical library in which twelve iterative solvers are written once, against one problem interface, and run unchanged over seven execution backends. Around them: a numerics module with root finders, quadrature rules, ODE integrators and dense factorisations, each with its convergence order tested against theory; two problem families; a benchmark harness that resolves a declared matrix, resumes after interruption and records the provenance of every row; and three reports built from the measurements by script.

7.2 What was established

1. **Identical numerics across execution models is achievable, not merely approachable.** With deterministic reductions every shared memory backend at every worker count produces bit identical iterates, and the GPU sweeps match the CPU bit for bit as well. This is asserted by the test suite as exact equality across twelve solvers, two problem families, four backends and seven worker counts, and it required fixing the reduction chunk grid by problem size and disabling fused multiply add contraction on both host and device.
2. **The classical convergence theory holds to the digit.** The Jacobi spectral radius matches $\cos(\pi h)$; Gauss Seidel needs exactly half the iterations of Jacobi, as $\rho_{GS} = \rho_J^2$ requires; Young's closed form relaxation factor is a genuine minimum; optimal relaxation changes the growth order from $O(n^2)$ to $O(n)$; and conjugate gradient reaches the same order without being told a parameter. The five point stencil is second order accurate with the error constant $\pi^2/12$ predicted by the leading truncation term.
3. **Sequential methods do not parallelise, and saying so is more useful than hiding it.** Natural ordering Gauss Seidel keeps exact semantics on every backend, including across MPI ranks, and shows essentially no speedup. Red black ordering is what buys the parallelism, at a measured cost of 2.6 percent more iterations.
4. **A GPU speedup on a bandwidth bound kernel is mostly the memory system.** Decomposed through (5.1), the observed advantage separates into a large factor that is the difference in measured bandwidth and a small factor that is the difference in how well each device is used. Reporting the undecomposed ratio would have been accurate and misleading.

7.3 What was not established

Stated as plainly as the conclusions, because a limitation left implicit reads as a claim.

- **The performance versus efficiency core split could not be identified from inside the guest.** WSL2 synthesises a homogeneous topology and affinity binds to a virtual processor. The knee is reported from the aggregate scaling curve instead, which is a weaker claim than originally intended and the strongest the environment supports.
- **Nothing is established about compute bound problems.** Every conclusion about devices here concerns bandwidth bound stencil work. A dense factorisation would rank the hardware differently.
- **Nothing is established about single solve latency,** power, cost, or programmer effort. Transfer time is reported separately rather than folded in; the rest were not measured and are not claimed.
- **The iteration counts at the largest grids are extrapolated,** not measured, for the methods whose counts grow like n^2 . The extrapolation rests on closed form rates that were verified at the sizes where measurement was affordable, and the report says which numbers are which.

7.4 Further work

Preconditioned conjugate gradient. The symmetric splittings are already implemented and symmetric Gauss Seidel and SSOR are exactly the preconditioners the method wants. Combining them would be a small amount of code and would demonstrate the point of the symmetric variants, which currently exist in the zoo without the application that motivates them.

Multigrid. Every component is present: smoothers in the relaxation sweeps, an exact coarse solve in the LU routine, and the grid structure in the Poisson problem. Multigrid is the method that makes the iteration count independent of the grid entirely, and adding it would put the $O(n^2)$ and $O(n)$ results of this report in their proper context as the two things it improves on.

A local slab MPI decomposition. Each rank currently allocates the whole grid. The communication volume is already correct, so this would not change any timing reported here, but it would remove the memory ceiling that keeps the distributed runs below the sizes the device comparison uses.

Bare metal measurement of the core knee. The one objective the environment prevented. Running the same sweep outside a hypervisor would settle whether the knee in the aggregate curve coincides with the performance core count, which is currently inferred rather than confirmed.

Mixed precision. The stencil sweeps are bandwidth bound, so halving the width of the iterate would come close to halving the time. Whether the resulting convergence penalty is worth it is exactly the kind of question this library's separation of iteration count from per iteration cost is built to answer.

Bibliography

- [1] Richard P. Brent. *Algorithms for Minimization without Derivatives*. Prentice Hall, Englewood Cliffs, 1973.
- [2] Richard L. Burden, J. Douglas Faires, and Annette M. Burden. *Numerical Analysis*. Cengage Learning, Boston, 10 edition, 2016.
- [3] Philip J. Davis and Philip Rabinowitz. *Methods of Numerical Integration*. Academic Press, Orlando, 2 edition, 1984.
- [4] J. R. Dormand and P. J. Prince. A family of embedded runge kutta formulae. *Journal of Computational and Applied Mathematics*, 6(1):19 to 26, 1980.
- [5] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, Baltimore, 4 edition, 2013.
- [6] Georg Hager and Gerhard Wellein. *Introduction to High Performance Computing for Scientists and Engineers*. CRC Press, Boca Raton, 2010.
- [7] Ernst Hairer, Syvert P. Nørsett, and Gerhard Wanner. *Solving Ordinary Differential Equations I: Nonstiff Problems*, volume 8 of *Springer Series in Computational Mathematics*. Springer, Berlin, 2 edition, 1993.
- [8] John D. McCalpin. Memory bandwidth and machine balance in current high performance computers. *IEEE Computer Society Technical Committee on Computer Architecture Newsletter*, page 19 to 25, 1995.
- [9] Michael McCool, Arch D. Robison, and James Reinders. *Structured Parallel Programming: Patterns for Efficient Computation*. Morgan Kaufmann, Waltham, 2012.
- [10] Message Passing Interface Forum. *MPI: A Message Passing Interface Standard, Version 4.0*. University of Tennessee, Knoxville, 6 2021.
- [11] OpenMP Architecture Review Board. *OpenMP Application Programming Interface, Version 6.0*. OpenMP Architecture Review Board, 11 2024.
- [12] Yousef Saad. *Iterative Methods for Sparse Linear Systems*. Society for Industrial and Applied Mathematics, Philadelphia, 2 edition, 2003.
- [13] Jonathan Richard Shewchuk. An introduction to the conjugate gradient method without the agonizing pain. Technical Report CMU-CS-94-125, School of Computer Science, Carnegie Mellon University, Pittsburgh, 1994.
- [14] Lloyd N. Trefethen and David Bau. *Numerical Linear Algebra*. Society for Industrial and Applied Mathematics, Philadelphia, 1997.

- [15] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65 to 76, 2009.
- [16] David M. Young. *Iterative Solution of Large Linear Systems*. Academic Press, New York, 1971.